MultiFuel

Quantified Project and Free Deployment Summary

A multi-station petrol pump management application for Owners, Managers, and duty staff. This report summarizes implemented workflows, measured codebase size, verification results, and the free-tier deployment path.

Project facts	Verified value
Technology	Django 5.2 · Python 3.11 · SQLite local · PostgreSQL hosted
Application modules	9 Django modules
Data models	33 across application modules
HTML templates	52
Application URL patterns	90, excluding Django admin
Automated tests	134 passed in the full test suite
Prepared	06 November 2026
Hosting target	Render free web service + external free PostgreSQL

Delivery status

The application implementation and free-tier deployment blueprint are prepared. The repository has been published, but hosting is not live yet: the owner must create a free external PostgreSQL database and apply the Blueprint from the Render account.

Implemented application capabilities

Accounts and station setup

- **3 roles:** Owner, Manager, and DSMS staff with role-protected dashboards and station scoping.
- Registration approval, account activation checks, password reset, and secure sign-out.
- Station profiles, Manager/DSMS assignments, fuel types, tanks, nozzles, prices, and lubricants.

Fuel and duty operations

- Fuel deliveries, tank-capacity checks, immutable stock ledger, low-stock indicators, and approval-based stock adjustments.
- Shift reports with meter readings, testing deductions, fuel-price snapshots, and lubricant sales.
- Submission, verification, correction, resubmission, approval, and immutable review history.

Reconciliation, credit, people, and expenses

- Cash denominations, UPI/cards, credit sales, advances, OTP settlements, report expenses, variances, and authorized adjustments.
- Credit customers, report-linked fuel sales, credit limits, repayments, statements, and overdue reporting.
- Attendance, approved leave, employee advances, payroll, printable payslips, and salary expense posting.
- Station expenses, Owner decisions, and accounting summaries.

Analytics, exports, and notifications

Capability	Measured coverage
Analytics reports	4: sales/expenses, inventory, duty/reconciliation, workforce
Export formats	3: CSV, Excel (.xlsx), PDF
Report-format combinations	12
Notification areas	6 workflow categories
Notification channels	In-app inbox; email depends on provider configuration

Email status is identified as console-only when an email backend only logs messages; such a message is not represented as delivered.

Verification and deployment

Verification	Result
Full automated test suite	134 passed
Django system check	Passed
Migration consistency	No ungenerated migrations
Render runtime plan	Free web service; no Render database provisioned

Free deployment steps

- Create a free PostgreSQL database with a provider such as Neon and copy its pooled connection string.
- In Render, connect the GitHub repository and apply the root `render.yaml` Blueprint.
- Enter `DATABASE_URL` and the initial Owner username, email, and strong password directly in Render's environment settings. Do not share or commit those credentials.
- The build runs migrations, creates the first approved Owner once, and collects static assets. After the first successful build, remove the `INITIAL_OWNER_*` variables.
- Use the Render-provided HTTPS hostname to sign in and create stations, staff, and fuel setup.

Free-tier limits and data caution

- Render free web services sleep after 15 minutes without traffic; waking can take about one minute.
- The web filesystem is ephemeral. SQLite and uploaded local files are not durable and must not hold live business records.
- Render free Postgres expires after 30 days, so the Blueprint uses an external database. External free database providers have quotas and availability limits.
- Render free services block common SMTP ports; password-reset and notification email is not delivered by this demo configuration.
- Free hosting is for demos, not dependable business use. Configure backups, monitoring, email delivery, and a production hosting plan before relying on live business data.

Financial reporting limitation

The accounting summary is not a full profit statement; it does not deduct fuel purchase costs, taxes, or other liabilities.

Counts are measured from the current project code. Test totals are from the latest full test run. This report is not a production-readiness or financial certification.